Using handshaking protocols

As pointed out previously, protocol blocks are implemented exactly as they are written, while internal behavioral code can be directed to be implemented in any number of ways. One architecture might implement a multiplication operation in one cycle, and another could save hardware and implement it in eight clock cycles. Because of this, it is difficult to write a good, multipurpose testbench without some knowledge of when the design will be ready to accept new input values or has valid data on its output ports.

In order for a testbench to know when to create input values and when to read the correct output values, some additional ports must be added. For each input port, an "input ready" output port must be added to tell the testbench that the design is ready for new data. Also, an "input valid" input port must be added so the testbench can tell the design that it can begin to read the values on the port.

Conversely, each output port also needs two ports added: an "output valid" output port, telling the testbench that valid data is on the port, and an "output ready" input port, telling the design that the testbench is ready to read another value on the port. This is called *ready-valid handshaking*, and once it is added any architecture generated by Cynthesizer can be tested with a single testbench. This greatly enhances verification and makes it much more efficient. Ready-valid handshaking is recommended if your design is going to be pipelined (in which case the output from the design for a specific input may be produced any number of clock cycles later, dependent on design constraints and synthesis directives).

To add ready-valid handshaking to a design, begin by declaring the ready and valid ports for each input and output. The `mymod SC_MODULE` declaration might look like this:

```
SC_MODULE( mymod ) {
  sc_in< bool >          clk;
  sc_in< bool >          reset;
  sc_in< sc_uint<4> >    data_in;
  sc_out< bool >         data_in_rdy;  // Input ready handshake
  sc_in< bool >          data_in_vld;  // Input valid handshake
  sc_out< sc_uint<8> >   data_out;
  sc_in< bool >          data_out_rdy;  // Output ready handshake
  sc_out< bool >         data_out_vld;  // Output valid handshake
  …
```

```
};
```

Now, all operations on input or output ports must include loops that wait until the ready-valid handshaking conditions are met. The protocol block that reads values through the input port might look like this:

```
{
  CYN_PROTOCOL( "read" );
  sc_uint<6> i = 0;
  do {
    data_in_rdy = 1;
    while (!data_in_vld)
      wait(1);
    BUFx[i] = data_in.read();
    wait(1);
    i++;
  } while( i != 32 );
  data_in_rdy = 0;
}
```

Here the `for` loop is replaced with a `do-while` loop that performs 32 iterations. The loop begins by asserting `data_in_rdy` high. This tells the testbench that the `mymod SC_MODULE` is ready to read new values on the `data_in` input port. The design then sits in a `while` loop waiting until the testbench acknowledges that it has placed a value on the port (by asserting `data_in_vld`). The input port is read, the design waits one cycle as the original protocol stated, the iteration counter is advanced, and the `do-while` loop is repeated. This handshake continues for each input value to be read. When the loop is complete, the `data_in_rdy` output is reset, telling the testbench that the design is busy processing the input values that were just read.

For the output protocol, handshaking ports are introduced into the design like this:

```
{
  CYN_PROTOCOL( "write" );
  sc_uint<6> i = 0;
  do {
    data_out = BUFy[i];
    data_out_vld = 1;
    do {
      wait(1);
    } while(!data_out_rdy);
    i++;
```

```
  while( i != 32 );
  data_out_vld = 0;
}
```

Again, the `for` loop is replaced with a `do-while` loop. The loop begins by making the output assignment. The design asserts `data_out_vld`, telling the testbench that valid data is on the port. The design then waits in a `do-while` loop for the testbench to acknowledge that it is ready to read this output value (by asserting `data_out_rdy`). The iteration counter is incremented and the loop repeats for all 32 output values. When the loop is complete, the design resets `data_out_vld`, telling the testbench that it is done producing values.

Using protocol blocks to register inputs

In addition to establishing a fixed protocol for all input and output interfaces, protocol blocks are also useful for controlling the registering of input signals. The style in which they are written can determine whether an input is registered or not. Consider this protocol block:

```
{
  CYN_PROTOCOL( "io_behv" );
  in1_x = in1.read();
  in2_x = in2.read();
  out1.write( in1_x && in2_x );
  wait();
}
```

Here some input values are read and then immediately used to determine the value of system output `out1`. A `wait()` statement is present afterward, but because of the way the protocol block is written, the input values are not registered when they are used in the design. To register the system inputs, simply move the `wait()` statement up so it appears immediately after the reading of the inputs, as follows:

```
{
  CYN_PROTOCOL( "io_behv" );
  in1_x = in1.read();
  in2_x = in2.read();
  wait();
  out1.write( in1_x && in2_x );
}
```

The presence of the `wait()` statement ensures that the input values are registered before being used elsewhere in the design.

Setting an input delay to allow use of inputs without registers

If Cynthesizer knows how far into each clock cycle the design's inputs can be expected to be stable, it can safely schedule operations that use those inputs during the remainder of the clock cycle. If it does not know the input delay, it must register each input before using since it doesn't know how much time is available in which to schedule operations.

The most common method of specifying an input delay is to define the delay for all input ports in the design using the `CYN_DEFAULT_INPUT_DELAY` directive. If the external module that writes these inputs has registered outputs, then a safe value for the input delay can be derived from the *clk->q* delay for registers in the target library. The following example shows how to use `CYN_DEFAULT_INPUT_DELAY`:

```
SC_MODULE(dut) {
   sc_in_clk clk;
   sc_in<bool> rst;
   sc_in<bool> enable;
   sc_in< sc_uint<16> > in_data;
   sc_in<bool> out_busy;
   sc_out<bool> out_vld;
   sc_out< sc_uint<16> > out_data;


   SC_CTOR(dut) {
      // Declare that all inputs will be stable 0.5 ns after
      // the start of each clock cycle.  This value may be the
      // clk->q delay for the upstream module.
      CYN_DEFAULT_INPUT_DELAY( 0.5, "input delay" );

      SC_CTHREAD(t,clk.pos());
      reset_signal_is( rst, 0 );
   }
   void t()
   {
      while (1) {
         // Read an input.
```

```
        bool was_vld;
        sc_uint<16> din, q;
        {  CYN_PROTOCOL("read");
           wait();
           was_vld = enable.read();
           din = in_data.read();
        }

        // Calculate.
        sc_uint<16> q = f(din);

        // Write an output.
        {  CYN_PROTOCOL("write");
           do { wait(); } while (out_busy.read());
           out_vld = was_vld;
           out_data = q;
        }
    }
};
```

The `CYN_DEFAULT_INPUT_DELAY` directive in the module's constructor will apply to each `SC_CTHREAD` in the module. To affect the timing of inputs in an individual thread, the directive can be placed it the thread's function instead.

Since the timing of each input is known, Cynthesizer will be able to schedule calculations in the `f()` function based on the `in_data` input in the same cycle that the input was read. This can reduce latencies, and eliminate the requirement for an input register.

Timing values for individual inputs can also be specified using the `CYN_INPUT_DELAY` directive. The default delay value may also be specified via the `--default_input_delay` command line option.

Declaring inputs as stable to avoid any registering

It is common for some design inputs to be registered by another module for well defined periods of time. These are called *stable inputs.* Cynthesizer will allow stable inputs to be used in calculations directly without inferring a register. The `CYN_STABLE_INPUT` directive can be used identify stable inputs, and to identify the region in which they can be expected to be stable.

In the following example, inputs `reg1` and `reg2` can be assumed to be held in registers by an external module for the time between the first `CYN_PROTOCOL` block and the last `CYN_PROTOCOL` block:

```
SC_MODULE(dut)
{
   sc_in_clk clk;
   sc_in<bool> rst;
   sc_out<bool> rdy_in;
   sc_in<bool> vld_in;
   sc_in< sc_uint<16> > reg1;
   sc_in< sc_uint<16> > reg2;
   sc_in<bool> rdy_out;
   sc_out<bool> vld_out;
   sc_out< sc_uint<16> > data_out;

   SC_CTOR(dut) {
      CYN_DEFAULT_INPUT_DELAY(0.5,"input delay");
      SC_CTHREAD(t, clk.pos());
      reset_signal_is(rst, 0);
   }
   void t() {
      sc_uint<16> q;
      while (1) {
         // Wait for a valid input.
         {  CYN_PROTOCOL("read");
            do {wait();} while (!vld_in.read());
         }
         {  //Define a block during which reg1 and reg2 are stable.
            CYN_STABLE_INPUT( reg1, "" );
            CYN_STABLE_INPUT( reg2, "" );

            // Calculate the result in this block.
            q = f(reg1.read(),reg2.read());
         }
         // Write the result.
         {  CYN_PROTOCOL("write");
            vld_out = 1;
            data_out = q;
            do {wait();} while (busy_out.read());
            vld_out = 0;
         }
   }
}
```

Notice that the CYN_STABLE_INPUT directive appears at the top of a block delimited by curly braces (e.g. {}). This is a free-scheduling region. The directive implies that the reg1 and reg2 inputs will have a stable value at any time in that block, and that Cynthesizer can schedule reads of those inputs at any time during that block. This will prevent a register from ever being inferred for an input read in a CYN_STABLE_INPUT block.

Defining a CYN_STABLE_INPUT block immediately between two bounding CYN_PROTOCOL blocks is very common because it identifies the time during which the external module has agreed to keep the inputs registered. Notice that the reg1.read() and reg2.read() expressions do not appear in a CYN_PROTOCOL block.

The CYN_STABLE_INPUTS directive is also provided for cases where there are a large number of stable inputs must be identified. It allows you to specify the first and last inputs in a range. Any other inputs falling in that range in the SC_MODULE definition will also be considered stable. In this example CYN_STABLE_INPUTS would be used as follows:

```
{  //Define a block during which reg1 and reg2 are stable.
   CYN_STABLE_INPUTS( reg1, reg2, "" );
```

For more information, see “Specifying input delays” (page 429, *UG*).

Accessing external memories in free-scheduled regions

External memories are accessed through metaport classes that may either be generated by cynthWB, or handwritten in C++. Each memory read or write is scheduled as a fixed length transaction. These memory transactions can appear in either fixed scheduling or free scheduling regions. More often than not, they appear in free scheduling regions, mingled directly with the code of the algorithm.

For example:

```
SC_MODULE(dut)
{
   // Input and output signals.
   sc_in_clk clk;
   sc_in<bool> rst;
```

```
    sc_out<bool> rdy_in;
    sc_in<bool> vld_in;
    sc_in< sc_uint<16> > data_in;
    sc_in<bool> rdy_out;
    sc_out<bool> vld_out;
    sc_out< sc_uint<16> > data_out;

    // External memory port.
    RAM256X16::port<> mem;

    SC_CTOR(dut) {
       CYN_DEFAULT_INPUT_DELAY(0.5,"input delay");
       SC_CTHREAD(t, clk.pos());
       reset_signal_is(rst, 0);
    }
    void t() {
       sc_uint<16> a, q;
       while (1) {
          // Wait for a valid input.
          { CYN_PROTOCOL("read");
            do {wait();} while (!vld_in.read());
            a = data_in.read()
          }

          // Read 16 values from the memory, starting at 'a'
          // and sum them.
          q = 0;
          for ( int i=0; i<16; i++ ) {
            CYN_UNROLL(ON,"reads");
            q += mem[a+i];
          }
          // Write the result.
          { CYN_PROTOCOL("write");
            vld_out = 1;
            data_out = q;
            do {wait();} while (busy_out.read());
            vld_out = 0;
          }
    }
  }
```

In this example the `mem[a]` access will result in a fixed-length memory read transaction taking place. Because this transaction takes place in a free-scheduling

region, Cynthesizer can move the memory read. Cynthesizer will obey the following rules when scheduling the memory read:

- The memory read will take place after the address has been calculated, but before its output is used.
- Memory read operations can be overlapped as long as the operations are separated by a number of cycles equal or greater to the defined initiation interval of the memory read operation. Each memory interface defines its initiation interval, but it is commonly 1, in which case, a memory read can be started every cycle.
- The memory read will not be moved into or past a surrounding `CYN_PROTOCOL` block.

In this last limitation, the `CYN_PROTOCOL` blocks at the start and end of the free-scheduling region in which the memory reads appear create *boundaries* for the movement of the memory reads. By default, all `CYN_PROTOCOL` blocks are treated as boundaries. However, this default boundary behavior can be modified as follows:

- A specific point in a `CYN_PROTOCOL` block can be defined to be boundary using the `CYN_BOUNDARY` directive. For more information, see "CYN_BOUNDARY" (page 83, *RG*).
- The `flags` parameter to a `CYN_MEM_READ_TX` or `CYN_MEM_WRITE_TX` directive can have the `CYN_IGNORE_BOUNDARIES` flag added to it to prevent boundaries from limiting the movement of the operation. For more information, see "CYN_MEM_READ_TX / CYN_MEM_WRITE_TX / CYN_PROTOCOL_TX" (page 128, *RG*).

These rules affect both `cynthWB`-generated external memory port classes, and handwritten external memory port classes. For more information on `cynthWB`-generated external memory classes, see "Using external memories" (page 562, *UG*). For more information on defining memory transactions, see "CYN_MEM_READ_TX / CYN_MEM_WRITE_TX / CYN_PROTOCOL_TX" (page 128, *RG*).

Example of design code transformation

This section shows the transformation of an algorithm written in C to an `SC_MODULE` that can be synthesized using Cynthesizer. First, the raw C program is presented. Next, the algorithm is modified to use bit-accurate SystemC data types and to fit into the SystemC simulation environment. Then hardware detail such as I/O protocol is added and it is put into a form that can be synthesized.

Basic C++ algorithm

Following is a C++ program that sends a set of 16 values into a trivial filter function, which in turn returns a weighted average of the last 3 samples.

```
#include <cstdio>

static float shift[3];
static int coefs[3] = { 15,10,5 };
static int sum_of_coefs;
static int initialized = 0;
static int values[16] =
{128,244,13,16,18,41,23,171,128,129,223,41,18,16,17,4 };

int silly_filter_func( float input_data )
{
   if ( !initialized ) {
      shift[2] = shift[1] = shift[0] = input_data;
      sum_of_coefs = coefs[0] + coefs[1] + coefs[2];
      initialized = 1;
   } else {
      shift[2] = shift[1];
      shift[1] = shift[0];
      shift[0] = input_data;
   }
   return ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
(shift[2]*coefs[2]) ) / sum_of_coefs;
}

int main( int argc, char* argv[] )
{
   int i;

   for ( i=0; i<15; i++ ) {
      printf( "result = %d\n", silly_filter_func( values[i] ) );
   }
}
```

C++ algorithm using SystemC constructs

Here is the same algorithm embodied in a C++ function using SystemC data. The `int` and `float` data types used in the C function have been changed to SystemC `sc_uints` with appropriate bit widths to achieve bit-accuracy.

```
#include <systemc.h>

static sc_uint<8> shift[3];
static sc_uint<4> coefs[3] = { 15,10,5 };
static sc_uint<6> sum_of_coefs;
static bool       initialized = 0;
static sc_uint<8> values[16] =
{128,244,13,16,18,41,23,171,128,129,223,41,18,16,17,4 };

sc_uint<8> silly_filter_func( sc_uint<8> input_data )
{
  if ( !initialized ) {
    shift[2] = shift[1] = shift[0] = input_data;
    sum_of_coefs = coefs[0] + coefs[1] + coefs[2];
    initialized = 1;
  } else {
    shift[2] = shift[1];
    shift[1] = shift[0];
    shift[0] = input_data;
  }
   return ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
(shift[2]*coefs[2]) ) / sum_of_coefs;
}

int sc_main(int argc, char* argv[])
{
   int i;
   for ( i=0; i<15; i++ ) {
      sc_uint<8> result = silly_filter_func( values[i] );
      printf( "result = %d\n", (int)result );
   }
   return 0;
}
```

Behavioral design ready for Cynthesizer

Here is the same functionality embodied in an SC_MODULE properly structured for synthesis by Cynthesizer.

The filter function has been placed into an SC_CTHREAD. The arguments to the function are passed into the module using a ready/valid handshake protocol. A similar protocol is used for passing the result out of the module. All I/O protocol has been placed in a code block delimited by a CYN_PROTOCOL directive.

```
#include <systemc.h>
#include <cynthhl.h>

SC_MODULE(silly_filter)
{
public:
  sc_in<bool>          reset;
  sc_out<bool>         input_ready;
  sc_in<bool>          input_valid;
  sc_in<sc_uint<8> >   input_data;
  sc_out<bool>         output_valid;
  sc_in<bool>          output_ready;
  sc_out<sc_uint<8> >  output_data;

  sc_in<bool>         CLK;

   SC_CTOR( silly_filter )
   {
     SC_CTHREAD(thread0, CLK.pos());
     reset_signal_is(reset, true);
   }

  void thread0();

protected:
   sc_uint<8>shift[3];
   sc_uint<6> sum_of_coefs;
   bool      initialized;
};

void silly_filter::thread0() {
   const sc_uint<4> coefs[3] = {15,10,5};
   sc_uint<8> sample;
```

```
sc_uint<8> result;

// execute reset logic
{  CYN_PROTOCOL( "Reset" );
   input_ready.write(false);
   output_data.write(0);
   output_valid.write(false);
   initialized = false;
   wait();
}
// multi-cycle initialization behavior
sum_of_coefs = coefs[0] + coefs[1] + coefs[2];

// main functionality
while(1) {
   // read input values
   { CYN_PROTOCOL("Read Loop");
      input_ready.write(true);
      do {
         wait();
      } while ( !input_valid.read() );
      sample = input_data.read();
      input_ready.write(false);
      wait();
   }

   // do the computation
   if ( !initialized ) {
      shift[2] = shift[1] = shift[0] = sample;
      initialized = true;
   }
   else {
      shift[2] = shift[1];
      shift[1] = shift[0];
      shift[0] = sample;
   }
   result = ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
             (shift[2]*coefs[2]) ) / sum_of_coefs;

   // write output values
   { CYN_PROTOCOL("Write");
      output_valid.write(true);
      output_data.write(result);
      do {
```

```
            wait();
        } while ( !output_ready.read() );
        output_valid.write(false);
    }
  }
};
```